

# SHUVAM PAL

+91 85830 54679    ishuvam.pal@gmail.com    Shuvam Pal    unichronic

## Education

Dayananda Sagar College of Engineering

Bengaluru, India

*B.E. Artificial Intelligence and Machine Learning, 3rd Year*

## Experience

### Bentham AI

Dec 2025 – Feb 2026

Backend Engineering Intern: Node.js, JavaScript, Go, Docker, GCP, GitHub Actions

Remote

- Built **backend automation for legal/compliance workflows** at [Bentham AI](#), combining browser execution, public-site API lookups, retries, **session recovery**, validation checks, and operator checkpoints to reduce manual filing time by **85%**.
- Modeled screenshots, API responses, generated drafts, form state, and failures as **recoverable workflow records**, making ambiguous external-system failures easier to debug and re-run.
- Implemented **AI-assisted drafting** while keeping generated outputs inspectable before deterministic execution, matching production workflows where model/provider output cannot be trusted blindly.
- Shipped Dockerized **Node.js/Go backend services** on **Google Cloud Run** with **GitHub Actions**, improving deployment velocity by **70%**.

### Google Summer of Code

May 2025 – Sept 2025

Mentee at Invesalio: Python, VTK, wxPython, NumPy, PyTorch, ONNX

Remote

- Integrated Python model-output workflows for **95 anatomical brain regions**, wiring preprocessing, PyTorch/TorchScript, ONNX/TinyGrad paths, generated artifacts, and correctness checks into Invesalio.
- Reduced expensive processing runtime by **87%** using async/parallel execution while debugging GPU/CPU paths, large data movement, generated masks, and output correctness.

## Open Source

[Contributions](#)

- Contributed maintainer-reviewed patches across **Kuadrant MCP Gateway**, **Kyverno**, **Invesalio**, **CRIU**, and **VideoLAN**, spanning backend reliability, security validation, parser robustness, compatibility fixes, and artifact correctness.

## Projects

**Swish Support Automation** | Python, FastAPI, PostgreSQL, Redis, Langfuse, pytest

[GitHub](#)

- Built **FastAPI support-agent services** with order, kitchen, fleet, trust, evidence, issue, and conversation context flowing through user-facing decisions.
- Implemented **provider fallback and cooldown handling** for LLM calls, request timeouts, Gemini fallback paths, local JSON/Langfuse traces, and deterministic fallbacks when provider output was unreliable.
- Added a live conversation-suite runner with request timeouts, retry-after handling, session clearing, pytest regressions, and response-quality checks for debugging multi-turn behavior.

**Hyoka Agent Reliability Layer** | Python, FastAPI, Pydantic, SQLAlchemy, PostgreSQL, OpenTelemetry

[GitHub](#)

- Built a **FastAPI agent reliability layer**: trace/event ingestion, eval/replay runs, failure mining, release gates, audit logs, project-scoped API keys, and worker leases.
- Implemented an **OpenAI-compatible proxy**, OTLP/OpenInference-style ingestion, Python SDK hooks for LLM/tool/retrieval events, artifacts, manifests, and promotions.
- Designed **typed metadata models** with SQLAlchemy/Alembic so agent behavior changes could be reproduced, compared, promoted, or rolled back safely.

**Postificus Content Distribution Engine** | Go, Echo, PostgreSQL, Redis, RabbitMQ, Docker, Prometheus

[GitHub](#)

- Built Go/Echo **REST APIs and worker services** with PostgreSQL persistence, Redis state/cache, RabbitMQ retry/DLQ jobs, Docker services, and Prometheus metrics.
- Kept long-running external workflows recoverable with saved state, retries, health checks, browser fallback execution, worker metrics, and clear UI status around publish/sync activity.

**Penny Lane Multi-Agent Workflow** | Python, Agno, Pydantic, SQLite, JSONL, React/Vite

[GitHub](#)

- Built a **multi-agent workflow** with analyst, debate, manager, risk-review, portfolio-approval, and evaluation stages wired through typed Pydantic state.
- Used SQLite checkpoints, JSONL/full-state traces, baseline comparisons, validation harnesses, and a React/Vite inspection UI to review pipeline outputs across runs.

## Technical Skills

**Languages:** Python, Go, TypeScript, JavaScript

**Frameworks:** FastAPI, Pydantic, SQLAlchemy, Node.js, Echo, React, Next.js, PyTorch

**Tools:** PostgreSQL, Redis, RabbitMQ, Docker, GCP Cloud Run, GitHub Actions, OpenTelemetry, Prometheus, Langfuse, ONNX